在提供适当署名的情况下，谷歌特此授权复制本文中的表格和图表，
仅限于用于新闻或学术作品。

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Researc
husz@google.com

Llion Jones*
Google Research
llion@google.com

AidanN.Gomez* †多伦多大学
aidan@cs.toronto.edu

Łukasz Kaiser*Google Brain
lukaszkaiser@google.com

伊利亚·波洛苏金* ‡
illia.polosukhin@gmail.com

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

摘要

主导的序列转导模型基于复杂的循环或卷积神经网络，这些网络包括编码器和解码器。表现最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构，Transformer，它完全基于注意力机制，摒弃了循环和卷积。在两个机器翻译任务上的实验表明，这些模型在质量上更优越，同时更具并行性，并且训练时间显著减少。我们的模型在WMT 2014 英语-德语翻译任务上达到了28.4 BLEU，比现有的最佳结果（包括集成模型）提高了超过2 BLEU。在WMT 2014 英语-法语翻译任务上，我们的模型在八块GPU上训练3.5天后，建立了一个新的单模型SOTA BLEU分数41.8，其训练成本只是文献中最佳模型的极小一部分。我们通过将其成功应用于英语依存句法分析任务，证明了Transformer对其他任务的泛化能力良好，无论是使用大量还是有限的训练数据。

*贡献均等。排序顺序随机。Jakob提议用自注意力机制替代循环神经网络，并开始评估这一想法。Ashish与Illia设计并实现了首批Transformer模型，并在这项工作的各个方面都发挥了关键作用。Noam提出了缩放点积注意力、多头注意力和无参数位置表示，并成为参与几乎所有细节的另一位关键人物。Niki设计、实现、调优和评估了我们原始代码库和tensor2tensor中的无数模型变体。Llion也实验了新型模型变体，负责我们最初的代码库、高效推理和可视化。Lukasz和Aidan花费无数漫长的日子设计和实现tensor2tensor的各个部分，替换了我们早期的代码库，极大地提升了结果，并极大地加速了我们的研究。

†在Google Brain期间完成的工作。

‡在Google Research期间完成的工作。

第31届神经信息处理系统大会（NIPS 2017），长滩，加利福尼亚州，美国。

1 简介

循环神经网络，特别是长短期记忆 [13] 和门控循环 [7] 神经网络，已被牢固确立为序列建模和转换问题（如语言建模和机器翻译 [35, 2, 5]）中的最先进方法。此后，人们持续投入大量努力以拓展循环语言模型和编码器-解码器架构 [38, 24, 15] 的边界。

循环模型通常沿着输入和输出序列的符号位置进行计算。将位置对齐到计算时间步，它们生成一个隐藏状态序列 h_t，作为先前隐藏状态 h_{t-1} 和位置 t 输入的函数。这种固有的顺序性质排除了在训练样本内部的并行化，这在较长的序列长度时变得至关重要，因为内存限制限制了跨样本的批处理。最近的工作通过分解技巧 [21] 和条件计算 [32]，实现了计算效率的显著改进，同时在后者的情况下还提高了模型性能。然而，顺序计算的 fundamental constraint 仍然存在。

注意力机制已成为各种任务中引人入胜的序列建模和转导模型的组成部分，允许在不考虑它们在输入或输出序列中距离的情况下对依赖关系进行建模 [2, 19]。但在几乎所有情况下 [27], ，这种注意力机制都与循环网络一起使用。

在这项工作中，我们提出了 Transformer，一种避免递归、完全依赖注意力机制来建立输入和输出之间全局依赖关系的模型架构。Transformer 允许显著更多的并行化，并且在经过在八块 P100 GPU 上训练仅十二小时后，即可达到翻译质量的新里程碑。

2 背景

减少顺序计算的目标也构成了扩展神经GPU[16], ByteNet [18] 和ConvS2S [9], 的基础，所有这些模型都使用卷积神经网络作为基本构建块，并行计算所有输入和输出位置的隐藏表示。在这些模型中，将两个任意输入或输出位置的信号关联起来所需的操作次数随着位置之间距离的增长而增加，对于ConvS2S是线性增长，对于ByteNet是对数增长。这使得学习远处位置之间的依赖关系 [12]变得更加困难。在Transformer中，这减少到了一个常数次的操作，尽管这会由于对注意力加权位置进行平均而导致有效分辨率降低，而我们通过第3.2节中描述的多头注意力机制来抵消这种影响。

自注意力机制，有时也称为序列内注意力，是一种关联单个序列不同位置的注意力机制，目的是计算序列的表示。自注意力机制已成功应用于多种任务，包括阅读理解、抽象摘要、文本蕴涵以及学习任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制，而非序列对齐的递归，并且已被证明在简单语言问答和语言建模任务上表现良好 [34]。

据我们所知，然而，Transformer 是第一个完全依赖自注意力机制来计算其输入和输出表示的转换模型，而不使用序列对齐的循环神经网络或卷积。在接下来的部分中，我们将描述 Transformer，阐述自注意力的动机，并讨论其相对于 [17, 18] 和 [9] 等模型的优点。

3 模型架构

大多数具有竞争力的神经序列转导模型都采用编码器-解码器结构 [5, 2, 35]。在此，编码器将输入序列的符号表示 $(x_1, ... x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, ... z_n)$。给定 $\mathbf{z}$，解码器随后逐个元素地生成输出序列 $(y_1, ... y_m)$。在每一步，模型都是自回归[10], 的，在生成下一个元素时将先前生成的符号作为附加输入进行消耗。

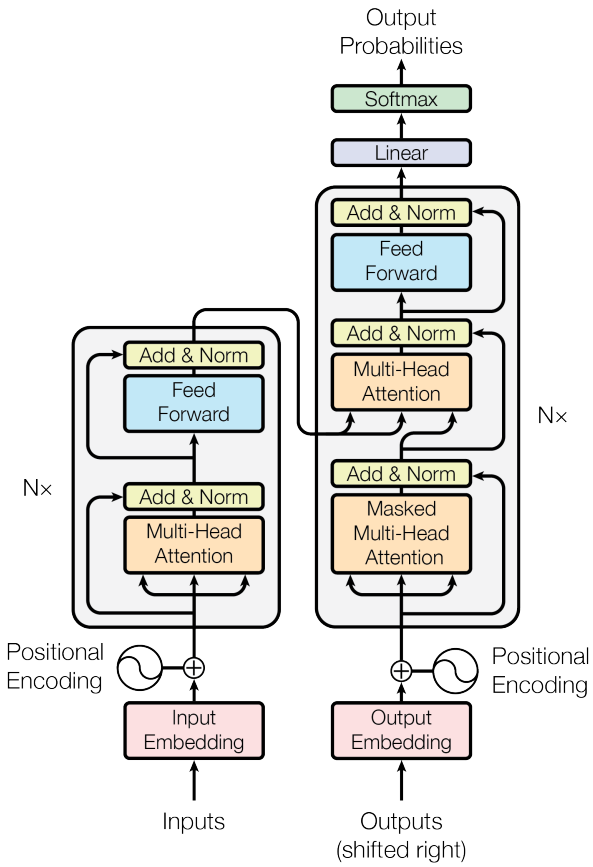


图1：Transformer - 模型架构。

Transformer 采用堆叠的自注意力机制和逐点全连接层来遵循这种整体架构，分别用于编码器和解码器，如图1的左半部和右半部所示。

3.1 编码器和解码器堆栈

编码器： 编码器由 $N = 6$ 个相同的层堆栈组成。每一层有两个子层。第一个是多头自注意力机制，第二个是一个简单、位置感知的全连接前馈网络。我们在每个子层周围使用残差连接 [11]，然后进行层归一化 [1]。也就是说，每个子层的输出是 $\text{LayerNorm}(x + \text{Sublayer}(x))$，其中 $\text{Sublayer}(x)$ 是子层本身实现的函数。为了方便这些残差连接，模型中的所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

解码器： 解码器也由 $N = 6$ 个相同的层堆栈组成。除了编码器层中的两个子层外，解码器还插入了一个第三个子层，该子层对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层周围使用残差连接，然后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩码，结合输出嵌入偏移一个位置的事实，确保位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 注意力

注意力函数可以被描述为将一个查询和一组键值对映射到一个输出，其中查询、键、值和输出都是向量。输出计算为值的加权求和

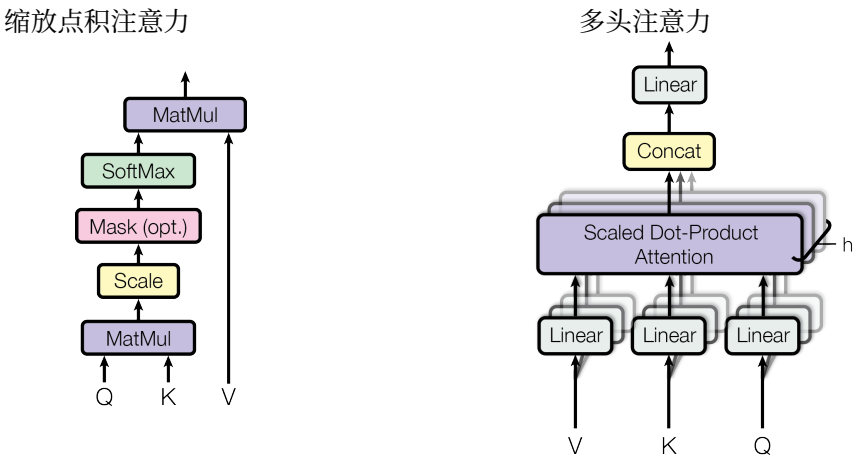


图2：左侧为缩放点积注意力。右侧为多头注意力由多个并行运行的注意力层组成。

其中分配给每个值的权重由查询与相应键的兼容性函数计算得出。

3.2.1 缩放点积注意力

我们将这种注意力称为“缩放点积注意力”（图2）。输入由维度为 d_k的查询和键，以及维度为 d_v的值组成。我们将查询与所有键的点积计算出来，除以 $\sqrt{d_k}$，然后应用softmax函数来获得值上的权重。

在实际中，我们同时在一组查询上计算注意力函数，将它们打包成一个矩阵 Q。键和值也被打包成矩阵 K 和 V。输出的矩阵计算如下：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

两种最常用的注意力函数是加性注意力 [2], 和点积（乘性）注意力。点积注意力与我们的算法相同，除了缩放因子 $\frac{1}{\sqrt{d_k}}$。加性注意力使用单隐藏层的前馈网络来计算兼容性函数。虽然理论上这两种方法的复杂度相似，但在实践中，点积注意力更快且更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值，这两种机制表现相似，但加性注意力在 d_k [3]值较大时优于无缩放的点积注意力。我们推测对于较大的d_k值，点积的绝对值会变得很大，将softmax函数推入梯度极小的区域 [4]。为了抵消这种效应，我们对点积进行缩放 $\frac{1}{\sqrt{d_k}}$。

3.2.2 多头注意力

与其使用具有 d_{model}-维键、值和查询的单个注意力函数，我们发现将查询、键和值线性投影 h 次到不同的、学习的线性投影中，分别映射到 d_k、 d_k 和 d_v 维是有益的。然后，我们对这些查询、键和值的投影版本并行执行注意力函数，生成 d_v-维

[4]为了说明为什么点积会变得很大，假设 q 和 k 的分量是均值为0、方差为1的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$的均值为0、方差为 d_k。

输出值。这些值被连接起来，再次投影，最终得到图2中所示的值。

多头注意力允许模型在不同位置同时关注来自不同表示子空间的信息。使用单个注意力头时，平均操作会抑制这种能力。

$$\begin{aligned}\text{MultiHead}(Q,K,V) &= \text{Concat}(\text{head}_1,...,\text{head}_\text{h})W^O \\ \text{where head}_\text{i} &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

其中投影是参数矩阵 $W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}$、 $W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}$、 $W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$。

在这项工作中，我们采用 $h = 8$ 个并行注意力层，或头。对于每一个，我们使用 $d_k = d_v = d_\text{model}/h = 64$。由于每个头的维度降低，总计算成本与完整维度下的单头注意力相似。

3.2.3 注意力在我们模型中的应用

Transformer 在三种不同方式中使用多头注意力：

- 在 "编码器-解码器注意力" 层中，查询来自前一个解码器层，记忆键和值来自编码器的输出。这允许解码器中的每个位置关注输入序列中的所有位置。这模拟了序列到序列模型（如 [38, 2, 9]）中的典型编码器-解码器注意力机制。

- 编码器包含自注意力层。在自注意力层中，所有的键、值和查询都来自同一位置，在本例中，来自编码器前一层的输出。编码器中的每个位置都可以关注编码器前一层的所有位置。

- 类似地，解码器中的自注意力层允许解码器中的每个位置关注解码器中所有位置，包括该位置本身。我们需要防止解码器中的左向信息流，以保持自回归属性。我们通过在缩放点积注意力内部屏蔽（设置为 $-\infty$）softmax 输入中对应非法连接的所有值来实现这一点。见图 2。

3.3 位置前馈网络

除了注意力子层外，我们编码器和解码器中的每一层都包含一个全连接前馈网络，该网络对每个位置分别且相同地应用。这由两个线性变换组成，中间有一个 ReLU 激活。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

虽然线性变换在不同位置上是相同的，但它们在层与层之间使用不同的参数。另一种描述方式是将其视为两个核大小为1的卷积。输入和输出的维度是 $d_\text{model} = 512$，内层的维度是 $d_{ff} = 2048$。

3.4 嵌入和Softmax

与其他序列转导模型类似，我们使用学习嵌入将输入标记和输出标记转换为维度为 d_model的向量。我们还使用通常的学习线性变换和softmax函数将解码器输出转换为预测下一个标记概率。在我们的模型中，我们在两个嵌入层和预softmax线性变换之间共享相同的权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_\text{model}}$。

表1：不同层类型的最大路径长度、每层复杂度和最小顺序操作数。 n 是序列长度， d 是表示维度， k 是卷积的核大小， r 是受限自注意力机制中的邻域大小。

层类型	每层复杂度	顺序操作数	最大路径长度
自注意力机制	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
自注意力机制（受限）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 位置编码

由于我们的模型不包含递归和卷积，为了让模型利用序列的顺序，我们必须注入一些关于序列中标记的相对或绝对位置的信息。为此，我们在编码器和解码器堆栈底部的输入嵌入中添加了"位置编码"。位置编码的维度与嵌入相同，以便两者可以相加。位置编码有很多选择，包括学习和固定的 [9]。

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

pos 表示位置， i 表示维度。也就是说，位置编码的每个维度都对应一个正弦波。波长从 2π 到 $10000 \cdot 2\pi$形成几何级数。我们选择这个函数是因为我们假设它能让模型更容易地通过相对位置来学习注意力机制，因为对于任何固定的偏移量 k， PE_{pos+k} 可以表示为PE_{pos} 的线性函数。

我们还尝试使用学习位置嵌入 [9] ，但发现这两个版本的结果几乎相同（见表3第(E)行）。我们选择正弦版本是因为它可能允许模型推广到训练中遇到的序列长度之外的序列长度。

4 为什么使用自注意力机制

在本节中，我们将自注意力层与通常用于将一个变量长度的符号表示序列(x_1，... x_n)映射到另一个等长度序列(z_1，... z_n)的循环层和卷积层进行比较，其中 $x_i, z_i \in \mathbb{R}^d$，例如典型序列转换编码器或解码器中的隐藏层。为了说明我们使用自注意力机制的原因，我们考虑了三个期望。

一个是每层的总计算复杂度。另一个是可以并行化的计算量，以所需的最小顺序操作数衡量。

三是网络中长期距离依赖之间的路径长度。学习长期距离依赖是许多序列转换任务中的一个关键挑战。影响学习这种依赖能力的一个关键因素是前向和后向信号在网络中必须穿越的路径长度。输入和输出序列中任何位置组合之间的路径越短，学习长期距离依赖就越容易 [12]。因此，我们还比较了由不同层类型组成的网络中任何两个输入和输出位置之间的最大路径长度。

如表1所示，自注意力层通过固定数量的顺序执行操作连接所有位置，而循环层需要 $O(n)$ 个顺序操作。在计算复杂度方面，当序列长度大于某个阈值时，自注意力层比循环层更快。

长度 n 小于表示维度 d，这通常是机器翻译中最先进的模型（如word-piece[38] 和 byte-pair [31] 表示）使用的句子表示的情况。为了提高涉及非常长序列的任务的计算性能，自注意力可以限制为仅考虑输入序列中围绕相应输出位置的中心大小为 r 的邻域。这将使最大路径长度增加到 $O(n/r)$。我们计划在未来的工作中进一步研究这种方法。

一个卷积层，其核宽度为 $k < n$，并不能连接所有输入和输出位置。要实现这一点，在连续核的情况下需要堆叠 $O(n/k)$ 个卷积层，而在扩张卷积的情况下则需要 $O(log_k(n))$ 个卷积层，从而增加网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，成本高出 k 倍。然而，可分离卷积 [6]，可以显著降低复杂度，降至 $O(k \cdot n \cdot d + n \cdot d^2)$。即便如此，可分离卷积的复杂度仍等于自注意力层和逐点前馈层的组合，这也是我们模型采用的方法。

作为附加好处，自注意力机制可以产生更易于解释的模型。我们检查了模型中的注意力分布，并在附录中呈现和讨论了相关示例。不仅单个注意力头能明显学会执行不同任务，许多注意力头似乎还表现出与句子句法和语义结构相关的行为。

5 训练

本节描述了我们模型的训练流程。

5.1 训练数据与批处理

我们在标准 WMT 2014 英语-德语数据集上训练，该数据集包含约 450 万个句子对。句子使用字节对编码 [3], 进行编码，其共享源-目标词汇表约有 37000 个词元。对于英语-法语，我们使用了规模显著更大的 WMT 2014 英语-法语数据集，包含 3600 万个句子，并将词元分割为 32000 个词元词汇 [38]。句子对按近似序列长度进行批处理。每个训练批次包含约 25000 个源词元和 25000 个目标词元的句子对集合。

5.2 硬件与时间表

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练我们的模型。对于论文中描述的基模型超参数，每个训练步骤约耗时 0.4 秒。我们训练基模型共 100,000 步或 12 小时。对于我们的大模型（见表 3 最下面一行描述），步骤耗时为 1.0 秒。大模型训练了 300,000 步（3.5 天）。

5.3 优化器

我们使用了 Adam 优化器 [20]，结合 $\beta_1 = 0.9$、$\beta_2 = 0.98$ 和 $\epsilon = 10^{-9}$。我们在训练过程中调整学习率，具体公式如下：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

这对应于在前 $warmup_steps$ 个训练步骤中线性增加学习率，之后则按步骤数的平方根倒数成比例降低。我们使用了 $warmup_steps = 4000$。

5.4 正则化

我们训练过程中采用三种正则化方法：

表2：Transformer在英语-德语和英语-法语newstest2014测试中，以更低的训练成本实现了比先前最先进模型更好的BLEU分数。

模型	BLEU		训练成本（FLOPs）	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT +RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att +PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT +集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S集成 [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer（基础模型）	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer（大模型）	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

残差Dropout 我们将dropout [33] 应用于每个子层的输出，然后将其添加到子层输入并进行归一化。此外，我们还将dropout应用于编码器和解码器堆栈中嵌入和位置编码的总和。对于基础模型，我们使用$P_{drop} = 0.1$的比率。

标签平滑 在训练过程中，我们采用了值为 $\epsilon_{ls} = 0.1$ [36]的标签平滑。这会降低困惑度，因为模型会变得不那么确定，但能提高准确率和BLEU分数。

6 结果

6.1 机器翻译

在 WMT 2014 英语-德语翻译任务上，表2中的大型 Transformer 模型（Transformer (big)）比之前报道的最佳模型（包括集成模型）多出 2.0 BLEU，创下了新的 BLEU 分数 28.4。该模型的配置列在表3的最底行。在 8 个 P100 GPU 上训练耗时 3.5 天。即使我们的基础模型也超越了所有之前发表的模型和集成模型，其训练成本远低于任何竞争模型。

在 WMT 2014 英语-法语翻译任务上，我们的大型模型取得了 41.0 的 BLEU 分数，超越了所有之前发表的单一模型，且训练成本不到 $1/4$ 之前最佳模型的 $1/4$ 。用于英语-法语翻译的 Transformer (big) 模型使用了 dropout 率 $P_{drop} = 0.1$，而不是 0.3。

对于基础模型，我们使用了通过平均最后5个检查点（每10分钟记录一次）得到的单个模型。对于大型模型，我们平均了最后20个检查点。我们使用了beam search，beam大小为4，并应用了长度惩罚 $\alpha = 0.6$ [38]。这些超参数是在开发集上进行实验后选择的。我们在推理时将最大输出长度设置为输入长度 + 50，但在可能的情况下提前终止 [38]。
表2总结了我们的结果，并将我们的翻译质量和训练成本与其他文献中的模型架构进行了比较。我们通过将训练时间、使用的GPU数量以及每个GPU的持续单精度浮点性能估计值相乘，来估计训练模型所需的浮点运算次数 [5]。

6.2 模型变体

为了评估 Transformer 不同组件的重要性，我们以不同方式调整了基础模型，并测量了在英语-德语翻译任务上开发集（newstest2013）上的性能变化

[5]我们使用了值2.8、3.7、6.0和9.5 TFLOPS，分别对应K80、K40、M40和P100。 7、6.0和9.5 TFLOPS ，分别。

表3：Transformer架构的变体。未列出的值与基础模型的值相同。所有指标均在英语-德语翻译开发集newstest2013上。列出的困惑度是按我们的字节对编码的每个词片段计算的，不应与每个词的困惑度进行比较。

	N	$d_{\text{模型}}$	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练步数	PPL (开发)	BLEU (dev)	参数 $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	位置嵌入而非正弦函数									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

我们使用了前一节中描述的集束搜索（beam search），但没有使用检查点平均（checkpoint averaging）。这些结果展示在表 3 中

在表3的行(A)中，我们变化了注意力头的数量以及注意力键和值的维度，同时保持计算量恒定，如第3.2.2节所述。虽然单头注意力比最佳设置差0.9 BLEU，但质量随着头数过多也会下降。

在表3的行(B)中，我们观察到减少注意力键大小 d_k 会损害模型质量。这表明确定兼容性并不容易，并且可能需要比点积更复杂的兼容性函数。我们进一步观察到在行(C)和(D)中，正如预期的那样，更大的模型效果更好，dropout在避免过拟合方面非常有帮助。在行(E)中，我们用学习位置嵌入 [9],替换了我们的正弦位置编码，并观察到与基础模型几乎相同的结果。

6.3 英语依存句法分析

为了评估Transformer是否可以泛化到其他任务，我们在英语依存句法分析上进行了实验。这项任务提出了特定的挑战：输出受强结构约束，并且显著长于输入。此外，RNN序列到序列模型在小数据环境下还没有能够达到最先进的结果 [37]。

我们使用 $d_{model} = 1024$ 在华尔街日报 (WSJ) 部分的宾夕法尼亚树库 [25]，上训练了一个 4 层 Transformer，大约有 40K 个训练句子。我们还以半监督方式训练了它，使用了大约 17M 个句子的较大高置信度 BerkeleyParser 语料库 [37]。对于 WSJ 单独的设置，我们使用了 16K 个 token 的词汇表，而对于半监督设置，我们使用了 32K 个 token 的词汇表。

我们仅进行了一小部分实验来选择 dropout、注意力机制和残差（第 5.4 节）、学习率以及束大小，这些实验是在第 22 个开发集上进行的，所有其他参数与英语-德语基础翻译模型保持不变。在推理过程中，我们将最大输出长度调整为输入长度 {v1}。我们使用了 21 的束大小和 {v2}

表4：Transformer在英语依存句法分析上泛化效果良好（结果来自华尔街日报第23部分）

解析器	训练	华尔街日报23 F1
Vinyals & Kaiser等（2014） [37]	仅华尔街日报，判别式	88.3
Petrov 等人 (2006) [29]	华尔街日报 仅，判别性	90.4
Zhu 等人 (2013) [40]	华尔街日报 仅，判别性	90.4
Dyer 等人 (2016) [8]	华尔街日报 仅，判别性	91.7
Transformer (4 层)	仅华尔街日报，判别性	91.3
Zhu 等人 (2013) [40]	半监督	91.3
黄 & 哈珀 (2009) [14]	半监督	91.3
McClosky 等人 (2006) [26]	半监督	92.1
Vinyals & Kaiser 等人 (2014) [37]	半监督	92.1
Transformer (4 层)	半监督	92.7
Luong 等人 (2015) [23]	多任务	93.0
Dyer 等人 (2016) [8]	生成式	93.3

incr将最大输出长度放宽至输入长度 + 300。我们使用了 21 的束大小和 $\alpha = 0.$ 3 for 既仅限于华尔街日报，也适用于半监督设置。

表4中的结果表明，尽管缺乏针对特定任务的微调，我们的模型表现出乎意料地好，其结果优于所有先前报道的模型，唯一的例外是循环神经网络语法 [8]。

与 RNN 序列到序列模型不同，Transformer 甚至在仅使用 WSJ 训练集（40K 句）进行训练时，也优于 Berkeley 解析器 [29] 。

7 结论

在这项工作中，我们提出了 Transformer，这是第一个完全基于注意力的序列转换模型，用多头自注意力机制取代了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度比基于循环层或卷积层的架构快得多。在 WMT 2014 英语-德语和 WMT 2014 英语-法语翻译任务上，我们取得了新的技术突破。在前者任务中，我们的最佳模型甚至优于所有先前报告的集成模型。

我们对基于注意力的模型的未来充满期待，并计划将它们应用于其他任务。我们计划将 Transformer 扩展到涉及文本以外的输入和输出模式的问题，并研究局部、受限的注意力机制，以高效处理图像、音频和视频等大型输入和输出。使生成过程更少依赖顺序是另一个研究目标。

我们用于训练和评估模型的代码可在 https://github.com/tensorflow/tensor2tensor 获取。

Ac致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 的富有成效的评论、修正和启发。 1 comments、修正和启发。

参考文献

[1] Jimmy Lei Ba、Jamie Ryan Kiros 和 Geoffrey E Hinton。层归一化。*arXiv* 预印本 *arXiv:1607.06450*，2016年。[2] Dzmitry Bahdanau、Kyunghyun Cho 和 Yoshua Bengio。通过联合学习对齐和翻译进行神经机器翻译。*CoRR*，abs/1409.0473，2014年。[3] Denny Britz、Anna Goldie、Minh-Thang Luong 和 Quoc V. Le。大规模探索神经机器翻译架构。*CoRR*，abs/1703.03906，2017年。[4] Jianpeng Cheng、Li Dong 和 Mirella Lapata。用于机器阅读的长短期记忆网络。*arXiv* 预印本 *arXiv:1601.06733*，2016年。

[5] Kyunghyun Cho、Bart van Merrienboer、Caglar Gulcehre、Fethi Bougares、Holger Schwenk 和 Yoshua Bengio。使用 RNN 编码器-解码器学习短语表示进行统计机器翻译。*CoRR*，abs/1406.1078，2014年。[6] Francois Chollet。Xception：使用深度可分离卷积的深度学习。*arXiv* 预印本 *arXiv:1610.02357*，2016年。[7] Junyoung Chung、Çaglar Gülçehre、Kyunghyun Cho 和 Yoshua Bengio。门控循环神经网络在序列建模上的经验评估。*CoRR*，abs/1412.3555，2014年。[8] Chris Dyer、Adhiguna Kuncoro、Miguel Ballesteros 和 Noah A. Smith。循环神经网络语法。在*NAACL* 会议录，2016年。[9] Jonas Gehring、Michael Auli、David Grangier、Denis Yarats 和 Yann N. Dauphin。卷积序列到序列学习。*arXiv* 预印本 *arXiv:1705.03122v2*，2017年。[10]Alex Graves。使用循环神经网络生成序列。*arXiv* 预印本 *arXiv:1308.0850*，2013年。[11] Kaiming He、Xiangyu Zhang、Shaoqing Ren 和 Jian Sun。用于图像识别的深度残差学习。在 *IEEE* 计算机视觉与模式识别会议论文集，第770–778页，2016年。[12] Sepp Hochreiter、Yoshua Bengio、Paolo Frasconi 和 Jürgen Schmidhuber。循环网络中的梯度流：学习长期依赖的困难，2001年。[13] Sepp Hochreiter 和 Jürgen Schmidhuber。长短期记忆。神经计算，9(8):1735–1780，1997年。[14] Zhongqiang Huang 和 Mary Harper。跨语言使用潜在标注自训练 PCFG 语法。在*2009* 年自然语言处理经验方法会议论文集，第832–841页。ACL，2009年8月。[15] Rafal Jozefowicz、Oriol Vinyals、Mike Schuster、Noam Shazeer 和 Yonghui Wu。探索语言建模的极限。*arXiv* 预印本 *arXiv:1602.02410*，2016年。[16] Łukasz Kaiser 和 Samy Bengio。主动记忆能替代注意力吗？在 神经信息处理系统进展（*NIPS*），2016年。[17] Łukasz Kaiser 和 Ilya Sutskever。神经 GPU 学习算法。在 国际学习表示会议（*ICLR*），2016年。[18] Nal Kalchbrenner、Lasse Espeholt、Karen Simonyan、Aaron van den Oord、Alex Graves 和 Kory Kavukcuoglu。线性时间神经机器翻译。*arXiv* 预印本 *arXiv:1610.10099v2*，2017年。[19] Yoon Kim、Carl Denton、Luong Hoang 和 Alexander M. Rush。结构化注意力网络。在国际学习表示会议，2017年。[20] Diederik Kingma 和 Jimmy Ba。Adam：一种随机优化方法。在 *ICLR*，2015年。[21] Oleksii Kuchaiev 和 Boris Ginsburg。LSTM 网络的分解技巧。*arXiv* 预印本 *arXiv:1703.10722*，2017年。[22] Zhouhan Lin、Minwei Feng、Cicero Nogueira dos Santos、Mo Yu、Bing Xiang、Bowen Zhou 和 Yoshua Bengio。结构化自注意力句子嵌入。*arXiv* 预印本 *arXiv:1703.03130*，2017年。[23] Minh-Thang Luong、Quoc V. Le、Ilya Sutskever、Oriol Vinyals 和 Lukasz Kaiser。多任务序列到序列学习。*arXiv* 预印本 *arXiv:1511.06114*，2015年。[24] Minh-Thang Luong、Hieu Pham 和 Christopher D Manning。基于注意力的神经机器翻译的有效方法。*arXiv* 预印本 *arXiv:1508.04025*，2015年。

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, 和 Beatrice Santorini. 构建大型英语标注语料库：宾夕法尼亚树库。 *计算语言学*, 19(2):313–330, 1993.[26] David McClosky, Eugene Charniak, 和 Mark Johnson. 有效的自训练用于句法分析。在 *NAACL 人类语言技术会议主会*, 页面 152–159。ACL, 2006 年 6 月.[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, 和 Jakob Uszkoreit. 可分解注意力模型。在*自然语言处理经验方法*, 2016.[28] Romain Paulus, Caiming Xiong, 和 Richard Socher. 用于抽象摘要的深度强化模型。 *arXiv 预印本 arXiv:1705.04304*, 2017.[29] Slav Petrov, Leon Barrett, Romain Thibaux, 和 Dan Klein. 学习精确、紧凑且可解释的树标注。在 *第 21 届计算语言学国际会议和 ACL 第 44 届年会*, 页面 433–440。ACL, 2006 年 7 月.[30] Ofir Press 和 Lior Wolf. 使用输出嵌入改进语言模型。 *arXiv 预印本 arXiv:1608.05859*, 2016.[31] Rico Sennrich, Barry Haddow, 和 Alexandra Birch. 使用子词单元进行罕见词神经机器翻译。 *arXiv 预印本 arXiv:1508.07909*, 2015.[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, 和 Jeff Dean. 惊人的大神经网络：稀疏门控专家混合层。 *arXiv 预印本 arXiv:1701.06538*, 2017.[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, 和 Ruslan Salakhutdi- nov. Dropout：一种简单的方法来防止神经网络过拟合。 *机器学习研究杂志*, 15(1):1929–1958, 2014.[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, 和 Rob Fergus. 端到端记忆网络。在 C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, 和 R. Garnett, 编辑的*神经信息处理系统进展 28*, 页面 2440–2448。Curran Associates, Inc., 2015.[35] Ilya Sutskever, Oriol Vinyals, 和 Quoc VV Le. 基于神经网络的序列到序列学习。在 *神经信息处理系统进展*, 页面 3104–3112, 2014.[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, 和 Zbigniew Wojna. 重新思考计算机视觉的 Inception 架构。 *CoRR*, abs/1512.00567, 2015.[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, 和 Hinton. 语法作为外语。在*神经信息处理系统进展*, 2015.[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, 等。谷歌的神经机器翻译系统：弥合人与机器翻译之间的差距。 *arXiv 预印本 arXiv:1609.08144*, 2016.[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, 和 Wei Xu. 用于神经机器翻译的具有快速前馈连接的深度循环模型。 *CoRR*, abs/1606.04199, 2016.[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, 和 Jingbo Zhu. 快速且精确的移位-规约成分句法分析。在 *第 51 届 ACL 年会（第一卷：长文）*, 页面 434–443。ACL, 2013 年 8 月。

注意力机制可视化

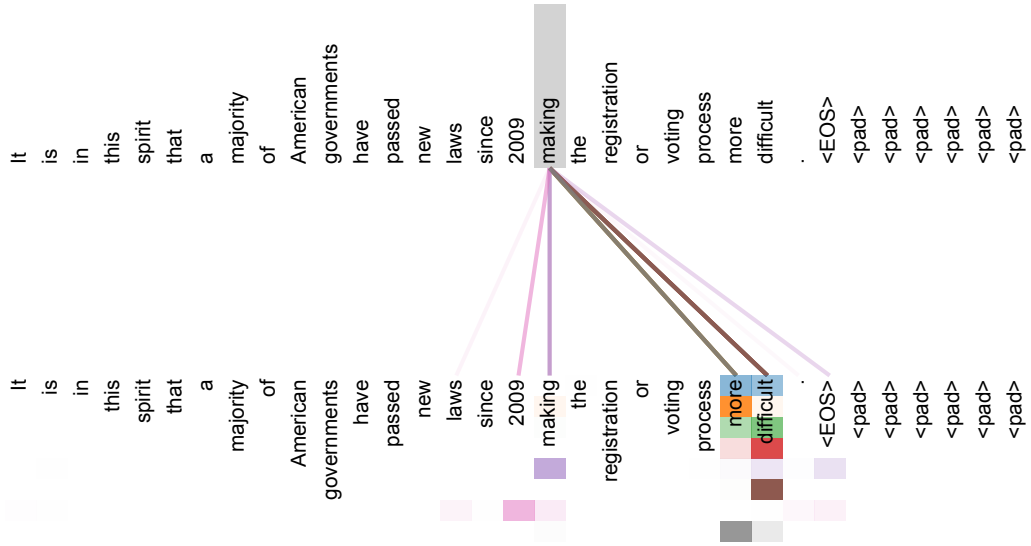


图3：展示了在编码器第5层（共6层）的自注意力机制中，长距离依赖的注意力机制示例。许多注意力头关注动词‘making’的远距离依赖，从而补全短语‘making...更困难’。此处仅展示单词‘making’的注意力。不同颜色代表不同的注意力头。建议彩色查看。

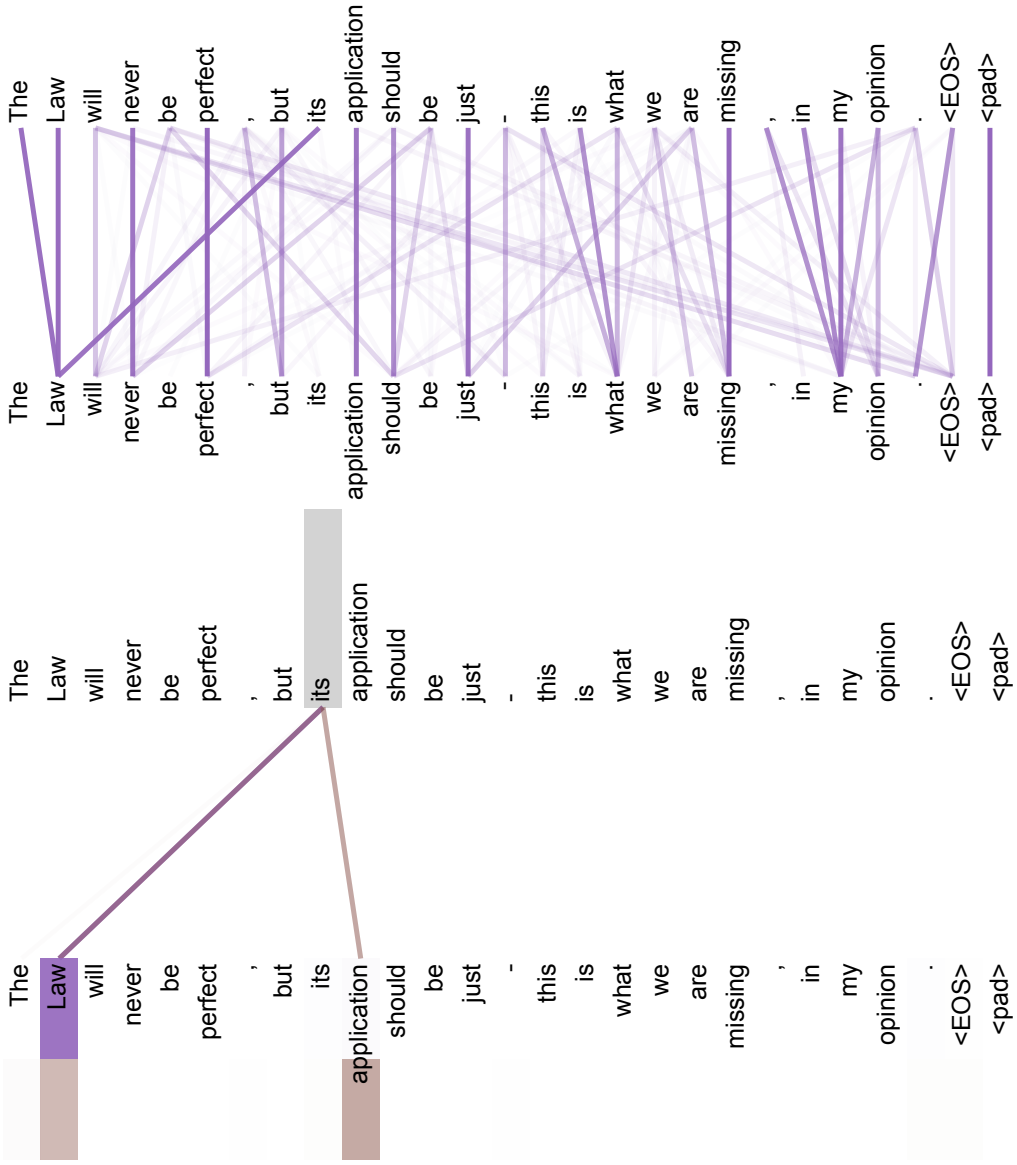


图4：两个注意力头，位于第6层的第5层，显然参与了指代消解。顶部：头5的完整注意力。底部：仅从单词‘其’对头5和6的注意力。注意，这个单词的注意力非常集中。

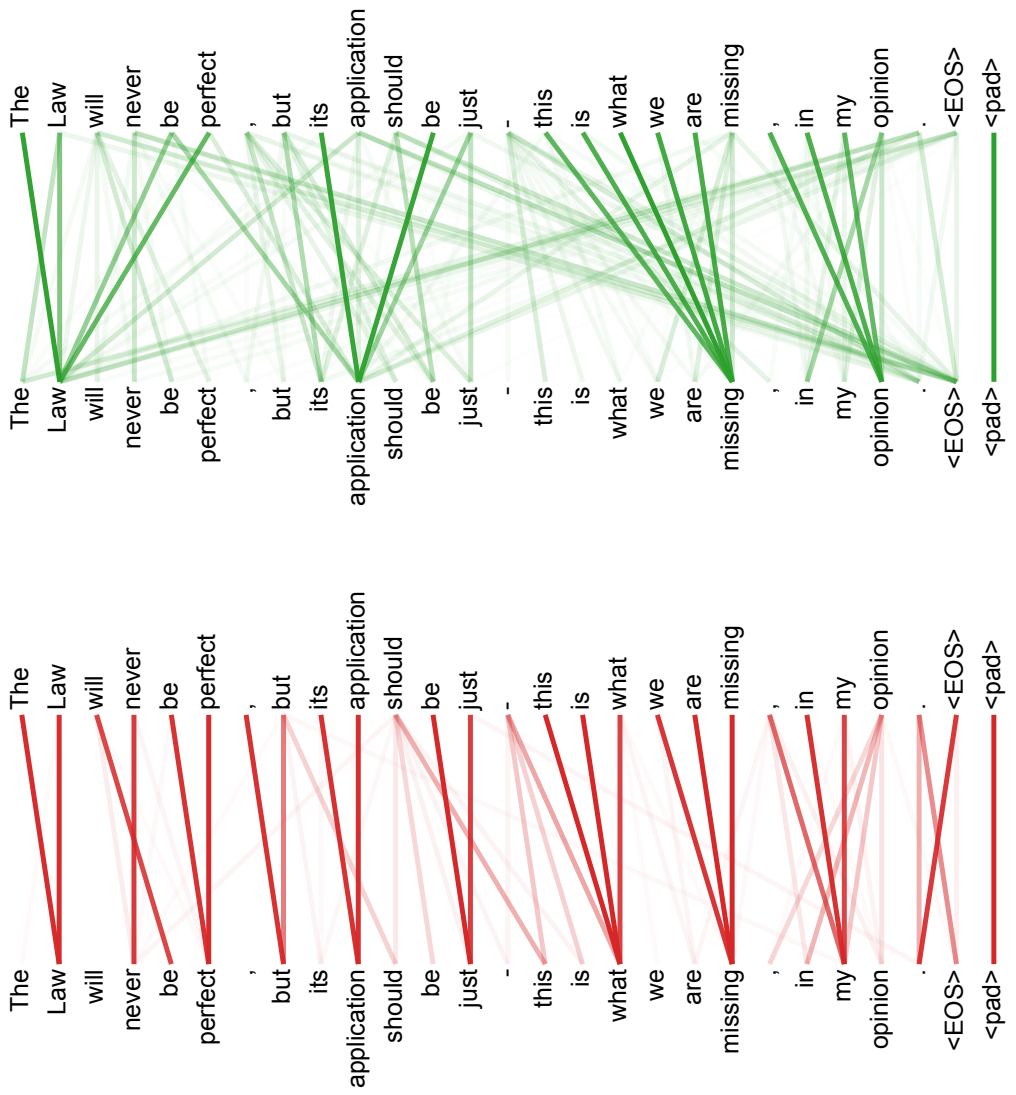


图5：许多注意力头表现出与句子结构相关的行为。我们在上面给出了两个这样的例子，来自编码器第5层（共6层）自注意力机制的不同头。这些头显然学会了执行不同的任务。